# Инфраструктура и измерения

Compose → readiness → воспроизводимость → время/RAM/диск/рост

# Что нужно знать

- Порядок db healthy → loader success → pipeline success → app. Почему depends_on с успешным завершением отличается от простого запуска процесса.
- Named volume базы и bind mounts для CSV, модели и измерений; почему секреты и сырые данные исключены из Git и Docker build context.
- Методика: три повторения, среднее и отдельные результаты, одна изменяемая величина, характеристики ноутбука.
- Полный запуск 31.10 s относится к готовым CSV и образам; это не время первого скачивания и сборки.
- Индекс сравнивается на одной выборке 300 клубов; 25/50/100% эксперименты — отдельная агрегация крупнейшей таблицы.
- Память — максимальное наблюдаемое working set из docker stats, а не непрерывно измеренный истинный пик.
- Что нужно подготовить к защите: данные и образы заранее, рабочий ноутбук, сохранённые результаты, PDF и двухминутное резервное видео.

# Что показать

1. Открыть docker-compose.yml и проследить зависимости и mounts без публикации паролей.
2. Показать README/MEASUREMENTS.md: машина, три прогона, диапазон условий и рост данных.
3. Объяснить отличие 31.10 s запуска от 10.872/0.0748 s запросов и от времени импорта одного CSV.

# Короткая речь на английском

> *Docker Compose coordinates four core services. The database must be healthy, loading and processing must finish successfully, and then the application starts. Source files, the model and measurement results use explicit mounts. We measured timing, observed memory, disk footprint and growth in separate experiments, each repeated three times. Complete cached-source startup averaged 31.10 seconds on an Apple M3 Pro with 18 GiB RAM. The selective index lookup improved by about 145 times, but that result applies to that query and sample. Caches were retained, image builds were excluded and brief memory peaks may have been missed.*

**Переход:** “Taken together, the system is reproducible on the measured setup, while the data and model limitations remain explicit.”

# Твой тайминг на защите

| Слайд | Блок | Время | Длительность |
|---|---|---|---|
| 8. Measure the whole system | 08 / Performance | 10:15–12:30 | 135 с |
| 9. Growth, memory and failure cases | 09 / Reliability | 12:30–14:00 | 90 с |

# Как закрывать экзамен

1. Твои слайды 8 и 9 (10:15–14:00) — ты закрываешь содержательную часть, последнюю минуту берёт Джакоппо, и твоя часть прямо соответствует обязательному пункту «производительность в числах».
2. Каждое число произноси с условиями: 31.10 с — среднее трёх прогонов с готовыми источниками и собранными образами, без скачивания и без сборки.
3. 145.4× называй как результат одного эксперимента на 300 выборочных запросах, а не как ускорение всего приложения. Этот вопрос задают почти всегда.
4. Память описывай методом: максимальная наблюдённая выборка docker stats с интервалом около двух секунд, не гарантированный пик.
5. Рост 25 / 50 / 100% объясняй нелинейностью: 4× строк дали 4.40× загрузки и 3.35× агрегации из-за фиксированных затрат, планов и кэшей.
6. Меняй один фактор за раз — скажи это вслух, это прямое требование протокола измерений из брифа.
7. В конце назови, что осталось: лицензия, независимый запуск на другом ноутбуке, резервное видео и фактический состав участников. Не заявляй их выполненными.

# Где смотреть в проекте

| Файл в проекте | Что в нём смотреть |
|---|---|
| `docker-compose.yml` | Порядок запуска: healthcheck базы → успешный loader → успешный pipeline → приложение. Профили для measure и legacy UI. |
| `docs/MEASUREMENTS.md` | Три прогона, железо (M3 Pro / 18 GiB), метод выборки памяти и все оговорки. Главный документ твоей части. |
| `docs/ARCHITECTURE.md` | Контейнеры как блоки, стрелки как файлы, строки, SQL и HTTP — структура для слайда архитектуры. |
| `docs/COURSE_REQUIREMENTS.md` | Таблица требований курса со страницами брифа и статусом каждого пункта. |

| Файл в проекте | Что в нём смотреть |
|---|---|
| `measure/benchmark.py` | Эксперимент с индексом: 300 одинаковых выборочных запросов, 145.4× — и почему это не ускорение всего приложения. |
| `scripts/measure_system.py` | Полный прогон с нуля: старт 31.10 с, loader 12.53 с, pipeline 9.7 с (средние трёх прогонов). |
| `scripts/report_measurements.py` | Сборка измерений в отчёт — показывает, что числа не вписаны руками. |
| `docs/AUDIT.md` | Найденные и исправленные дефекты: ответ на вопрос «как вы убедились, что это работает». |
| `docs/WORK_PACKAGES.md` | Пакеты работ, приёмочные условия и честный список незакрытых внешних пунктов. |
| `scripts/build_defense.py` | Сборка этого пособия: слайды, роли, PDF и проверка, что выступление ровно 900 секунд. |

# Практические задания

1. По схеме объясни, что произойдёт, если loader завершится с ошибкой, а база уже healthy.
2. Сравни 25% и 100%: строк в 4 раза больше, загрузка примерно в 4.40 раза дольше; объясни, почему это не универсальный закон.
3. Подними проект по README на другом ноутбуке и запиши наблюдения. Не объявляй эту проверку выполненной до реального запуска.

# Вопросы по направлению

**What starts when you run Docker Compose?**

Что запускает Compose?

The database becomes healthy, then the loader must finish successfully, then the processing pipeline must finish, and finally the application starts. The measurement service is on demand and the legacy interface uses an optional profile.

**What does the 31.10-second measurement include?**

Что входит в 31.10 секунды?

A mean over three complete starts with prepared source files and existing images: containers are recreated, sources reimported and the model retrained until the API is ready. Network acquisition and image builds are excluded and caches are not flushed.

**Does the index make the whole application 145 times faster?**

Всё приложение быстрее в 145 раз?

No. The measured gain is for 300 identical selective club lookups on the raw performance table, comparing index conditions. The frontend reads prepared outputs and a full GROUP BY has a different access pattern.

### Is your memory number a guaranteed peak?

Это гарантированный пик RAM?

It is the largest observed Docker working-set sample during three runs, sampled at roughly two-second intervals. Brief peaks may be missed. We report the method rather than label it a continuous or hardware-independent maximum.

### Why did four times as many rows not give exactly four times the query time?

Почему масштабирование не точно линейное?

Fixed costs, query plans, indexing, caches and hardware influence scaling. In this experiment four times the largest-table rows gave about 4.40 times the loading time and 3.35 times the aggregation time. That does not define a general law.

### What remains before a group submission?

Что ещё нужно для сдачи?

Confirm actual team membership and each person’s contribution, run an independent second-laptop setup, resolve the source licence, rehearse the presentation and record a backup demo. Equal author email counts alone do not demonstrate real group participation.

### What is the main result in one sentence?

Главный результат одним предложением?

The system is reproducible on the measured setup, while historical transfer spending alone has almost no linear relationship with next-season index change and the richer model adds only modest predictive value.

### What is the most important limitation?

Главное ограничение?

The data are retrospective and unevenly covered, the index is a proxy, and associations are not causal effects. Source licence and independent team/laptop validation also remain explicit unresolved requirements.

### How do goals for, goals against and goal difference work in the comparison?

Как в сравнении рассчитываются забитые, пропущенные голы и разница?

GF and GA are season totals rebuilt from completed home/away match scores. GD is GF − GA. They come from the optional football-data.co.uk match enrichment, not from the Champagne Index player totals.

### How are attacking and defensive ranks calculated?

Как рассчитываются атакующий и защитный рейтинги?

Within one league and season, attack rank orders goals scored per match from highest to lowest; defense rank orders goals conceded per match from lowest to highest. SQL RANK gives tied rates the same rank and leaves a gap after ties. They describe scores, not tactics or chance quality.

### Do we have xG or can we estimate it?

Есть ли в данных xG или можем ли мы его оценить?

The source has no provider xG field. The comparison instead shows observed shots and shots on target. A separate estimate multiplies shots on target by the league-season average goals-per-shot-on-target rate; it is an experimental volume proxy with no shot-location or chance-quality input, not xG.

### What is the difference between the Champagne Index and attack rank?

Чем атакующий рейтинг отличается от Champagne Index?

The Champagne Index is recorded player goals plus assists divided by a Transfermarkt matches proxy for a club-season. Attack rank uses actual completed league scorelines, sums team goals and ranks goals per match within the league-season. Their sources, coverage and units differ.

### How did you check the goalkeeper defensive totals?

Как вы проверили вратарские показатели по пропущенным?

We compared Transfermarkt goalkeeper goals-conceded totals with the independent match-result totals on 388 adequately covered top-five club-seasons. They match exactly in 366 (94.3%). Clean sheets match only 269 (69.3%), so the scoreline source remains the basis of league ranks.

### What does the defensive-spend analysis show?

Что показывает анализ расходов на защитников?

The Pearson correlation between defender transfer fees and next-season change in the goalkeeper-source conceded-goal rate is r=0.004047, p=0.426997, n=38,531. The sample does not support a useful linear association. It is descriptive, not causal. A separate match-level check finds teams with a red card concede 0.341 more goals on average in the same match, but match context confounds that association.

### Why keep the player-source defensive values separate from scoreboard goals?

Почему defensive-показатели по игрокам отделены от итогов матчей?

The sources use different records and denominators. Goalkeeper conceded totals agree with scorelines in 94.3% of adequately covered seasons, while clean sheets agree in 69.3%. The UI therefore uses match scores for GF, GA, GD and ranks; player-source totals remain a quality cross-check. Shots and corners are also observed match statistics, not chance-quality measures.

### Is the shot-based estimate the same as xG?

Оценка по ударам — это xG?

No. It multiplies a team’s shots on target by the league-season average goals per shot on target. It has no shot-location, distance, body-part, pressure or goalkeeper-position input. It is a rough volume benchmark and must be called an SOT-based proxy, not provider xG.

### What does the Poisson attack/defense rank tell us?

Что показывают Poisson-ранги атаки и обороны?

The model fits attack, defense and a shared home-advantage term to the selected league-season scores. Larger fitted attack and defense parameters mean stronger estimated attack and defense. Its ranks are descriptive and in-sample; the separate chronological RandomForest evaluation is the project’s held-out prediction check.

### Did the added defensive features improve the model?

Улучшили ли защитные признаки модель?

On the same chronological split, $R^2$ moves from 0.1271 with the five-feature baseline to 0.1502 after adding goalkeeper conceded rate, discipline and net spend; MAE moves from 0.4856 to 0.4788 on 5,585 held-out rows. The gain is modest, and it does not isolate a causal effect of defense.

**Do red cards cause more goals to be conceded?**

Красные карточки приводят к большему числу пропущенных?

The local sample shows a same-match difference of +0.341 conceded goals per team-match (1,619 sent-off vs 16,195 others; Welch p < 0.0001). This is observational: stronger opponents, match state and timing can affect both red cards and goals. It is not a causal estimate.

# Какие фразы не использовать

- 31 секунда включает скачивание 353 MiB и сборку образов.
- docker stats гарантирует истинный максимальный расход памяти.
- Одинаковое число коммитов подтверждает одинаковый личный вклад.

# Экзаменационный день

| Дата | Формат | Что требуется |
| --- | --- | --- |
| **12 ноября 2026**<br>CIDS Lecture 15, стр. 37–39 | Чекпоинт · 5 минут | Сквозное демо, один видимый результат и первое измеренное время полного прогона. Красивый интерфейс не требуется. |
| **20 / 26 ноября 2026**<br>CIDS Lecture 15, стр. 37–39 | Финальная защита · 15 минут + 5 минут вопросов | Говорят все реальные участники. Рекомендованный хронометраж: вопрос и данные 3 мин, архитектура 2, живое демо 3, измерения 3, проблемы 2, вклад 2. |

## Подготовка машины и материалов

1. За 30 минут: `docker compose up -d --build`, затем открыть `/api/health` и дождаться `ready=true` вместе с завершёнными loader и pipeline.
2. Открыть заранее четыре вкладки: Overview (Serie A 24/25), сравнение Inter / Milan, Lab и страницу Project с числами.
3. Открыть офлайн-копию пособия `defense/site/index.html` и `defense/site/downloads/Presentation.pdf` — они работают без сети и без базы.
4. Проверить, что скриншоты `defense/site/assets/*-desktop.png` на месте: это резерв, если Docker не поднимется.
5. Отключить уведомления, подключить питание, проверить зеркалирование экрана и масштаб шрифта в браузере.
6. Решить заранее, кто ведёт демо и кто держит таймер; таймер есть в `defense/site/presentation.html` (клавиша N — заметки, F — полный экран).

## Как отвечать на вопросы

- Отвечай тремя шагами: короткий ответ одним предложением → основание (файл или число) → ограничение.
- Каждое число называй с выборкой и единицей: «r = −0.79 на 388 клуб-сезонах в одном сезоне», а не «корреляция высокая».
- Не знаешь ответа — скажи, где он лежит, и не придумывай: «это в docs/MEASUREMENTS.md, метод измерения памяти описан там».
- Вопрос не по твоей части — передай одной фразой владельцу направления, не пересказывай чужую зону приблизительно.
- Никогда не превращай связь в причину: говори accompanies, is associated with, а не causes или improves.
- Если спросили про слабое место — отвечай прямо и первым. Признанное ограничение выглядит сильнее, чем защищаемая неточность.

## Неудобные вопросы: отвечай прямо и первым

| Тема | Вопрос | Ответ и где подтверждение |
|---|---|---|
| Лицензия первичного источника | Is your data openly licensed? | We pinned the source revision and verified headers and checksums, but we did not establish an explicit reuse licence. We document it as unresolved and we do not redistribute the raw data. Public downloadability is not a licence. `docs/SOURCES.md, docs/COURSE_REQUIREMENTS.md` |
| Четыре контрибьютора | Who did what in this project? | Describe only the work you can open and explain on screen. Git identities and role cards are not proof of independent contribution, and the repository says so. `docs/WORK_PACKAGES.md, README.md` |
| Проверка на другом ноутбуке | Did someone else reproduce it? | The run is reproducible on the measured machine. An independent fresh checkout on another laptop has not been recorded yet, so we do not claim it. `docs/MEASUREMENTS.md, docs/WORK_PACKAGES.md` |
| xG и прокси-метрики | Is this xG? | No. The source has no provider xG field. We show observed shots and shots on target, and a shots-on-target conversion proxy that we label experimental. Poisson attack and defense ratings are in-sample descriptions. `docs/GLOSSARY.md, pipeline/proxy_xg.py, pipeline/power_ratings.py` |

> Роль — учебная ответственность. Не заявляй личное авторство или самостоятельную проверку, если этого не было. Покажи ровно те изменения и команды, которые выполнял(а) лично.